

Conjugate Gradient Poisson Solver

Jie-Cheng Lin, Wen-Hung Luo, Ming-Chun Lin, I-Lun Tsai

Department of Physics
National Taiwan University

Jun 2025

Outline

1 Introduction

2 Principle

3 Implementation

- Implementation (CG method)
- CG Algorithm Parallelization

4 Performance

- Performance with Difference BC
- Performance with Difference Grid Size N
- Performance with and w/o Matrix-Free
- Wall-Clock Time vs. Number of cells
- Performance (other plots)
- Performance (other plots)
- Performance (other plots)

5 Q&A

Introduction

- Implement the Conjugate Gradient (CG) Method with parallel processing to Solve the 2D Poisson Equation.
 - The typical form of the equation is: $\nabla^2 \phi = f(x, y)$.
 - After discretization, it becomes a sparse linear system: $\mathbf{Ax} = \mathbf{b}$.
 - The CG method efficiently solves this system when $\mathbf{A}$ is symmetric and positive-definite.
- Performance Evaluation.
 - Measure execution time at different grid resolutions (number of cells), and analyze the scaling behavior (wall-clock time vs. grid size).
 - Compare the performance with the traditional SOR (Successive Over-Relaxation) method, including number of iterations to converge and execution time.

Principle

1 Construct the Discretized Grid and Matrix A

For example, use a five-point stencil to discretize the 2D Poisson equation.

2 Initialize $\mathbf{x}_0$, and compute the initial residual:

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$$

3 Set:

$$\mathbf{p}_0 = \mathbf{r}_0$$

Then perform the following iterations:

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k},$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k,$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k,$$

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k},$$

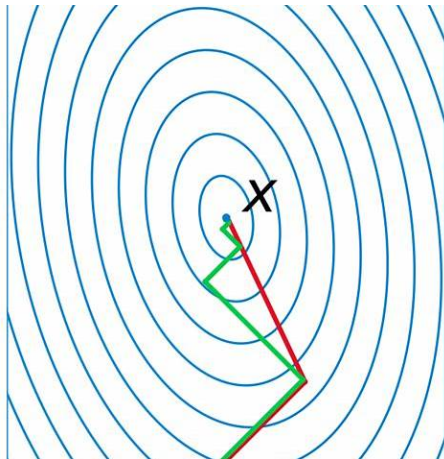
$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

4 Check for Convergence:

$$\|\mathbf{r}_{k+1}\| < \text{tolerance}$$

If the condition is met, stop the iteration.

Implementation (CG method)



Poisson equation

$$\nabla^2 f = g$$

Poisson equation

$$\nabla^2 f = g \qquad \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} \right) f = g$$

Poisson equation

$$\frac{\partial^2}{\partial x^2} f(x_i, y_i) = \frac{1}{h} \left(\frac{\partial}{\partial x} f(x_{i+\frac{1}{2}}, y_i) - \frac{\partial}{\partial x} f(x_{i-\frac{1}{2}}, y_i) \right)$$

Poisson equation

$$\frac{\partial^2}{\partial x^2} f(x_i, y_i) = \frac{1}{h} \left(\frac{\partial}{\partial x} f(x_{i+\frac{1}{2}}, y_i) - \frac{\partial}{\partial x} f(x_{i-\frac{1}{2}}, y_i) \right)$$

$$\frac{1}{h} \left(\frac{\partial}{\partial x} f(x_{i+\frac{1}{2}}, y_i) - \frac{\partial}{\partial x} f(x_{i-\frac{1}{2}}, y_i) \right) = \frac{1}{h} \left(\frac{1}{h} (f(x_{i+1}, y_i) - f(x_i, y_i)) - \frac{1}{h} (f(x_i, y_i) - f(x_{i-1}, y_i)) \right)$$

Poisson equation

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

Poisson equation

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

$$Ax = b$$

Shape resize

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16



1
2
3
4
5
6
7
⋮

Implementation (CG method)

Formulate A

1	2	3	4
N^2+1	N^2+2	N^2+3	N^2+4
$2N^2+1$	$2N^2+2$	$2N^2+3$	$2N^2+4$
$3N^2+1$	$3N^2+2$	$3N^2+3$	$3N^2+4$

⋮

$N^4-(N^2-1)$	$N^4-(N^2-2)$	$N^4-(N^2-3)$	$N^4-(N^2-4)$
---------------	---------------	---------------	---------------

...

N^2
$2N^2$
$3N^2$
$4N^2$

⋮

N^4

...

1
2
3
4
5
6
7

⋮

N^2

Implementation (CG method)

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

-4	0	0	0
0	-4	0	0
0	0	-4	0
0	0	0	-4

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

-4

...

1
2
3
4
5
6
7

⋮

N ²

Implementation (CG method)

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

-4	1	0	0
0	-4	1	0
0	0	-4	1
0	0	0	-4

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

-4

...

1
2
3
4
5
6
7

⋮

N ²

Implementation (CG method)

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

-4	1	0	0
1	-4	1	0
0	1	-4	1
0	0	1	-4

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

-4

...

1
2
3
4
5
6
7

⋮

N ²

Boundary condition

$$f(x_{i-1}, y_i) + f(x_{i+1}, y_i) + f(x_i, y_{i-1}) + f(x_i, y_{i+1}) - 4f(x_i, y_i) = h^2 g$$

$$\text{B.C 1: } f(x=0, y_i) = f(x=1, y_i) = 0 \quad \frac{\partial}{\partial n} f(x_i, y=0) = \frac{\partial}{\partial n} f(x_i, y=1) = 0$$

$$\text{B.C 2: } f(x=0, y_i) = f(x=1, y_i) = f(x_i, y=0) = f(x_i, y=1) = u(x, y)$$

Implementation (CG method)

$$\text{B.C 1: } f(x=0, y_i) = f(x=1, y_i) = 0 \quad \frac{\partial}{\partial n} f(x_i, y=0) = \frac{\partial}{\partial n} f(x_i, y=1) = 0$$

-4	1	0	0
1	-4	1	0
0	1	-4	1
0	0	1	-4

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

-4

...

1
2
3
4
5
6
7

⋮

N ²

Implementation (CG method)

$$\text{B.C 1: } f(x=0, y_i) = f(x=1, y_i) = 0 \quad \frac{\partial}{\partial n} f(x_i, y=0) = \frac{\partial}{\partial n} f(x_i, y=1) = 0$$

1	0	0	0
0	1	0	0
0	1	-4	1
0	0	1	-4

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

1

...

1
2
3
4
5
6
7

⋮

N^2

Implementation (CG method)

$$\text{B.C 1: } f(x=0, y_i) = f(x=1, y_i) = 0 \quad \frac{\partial}{\partial n} f(x_i, y=0) = \frac{\partial}{\partial n} f(x_i, y=1) = 0$$

1	0	0	0
0	1	0	0
0	0	-1	1
0	0	0	-1

⋮

0	0	0	0
---	---	---	---

...

0
0
0
0

⋮

1

...

1
2
3
4
5
6
7

⋮

N ²

Implementation (CG method)

B.C 1 :

Matrix (25 * 25):																								
Row 00	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 01	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 02	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 03	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 04	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 05	0	0	0	0	0	-1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 06	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Row 07	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
Row 08	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Row 09	0	0	0	0	0	0	0	0	1	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Row 10	0	0	0	0	0	0	0	0	0	0	-1	1	0	0	0	0	0	0	0	0	0	0	0	0
Row 11	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0	0	0
Row 12	0	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0	0
Row 13	0	0	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0	0	0	0
Row 14	0	0	0	0	0	0	0	0	0	0	0	0	0	1	-1	0	0	0	0	0	0	0	0	0
Row 15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	-1	1	0	0	0	0	0	0	0
Row 16	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0	0
Row 17	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1	0
Row 18	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	-4	1	0	0	0	1
Row 19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	-1	0	0	0
Row 20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
Row 21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
Row 22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
Row 23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
Row 24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Implementation (CG method)

B.C 2 :

```
Matrix (25 * 25):
Row 00 | 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
```

Implementation (CG method)

```
for (i=0;i<ROW-2;i++){  
    for (j=0;j<COL-2;j++){  
        b[IDX(i,j)] = dx*dx*func(i+1,j+1,dx,dy);  
    }  
}
```

Implementation (CG method)

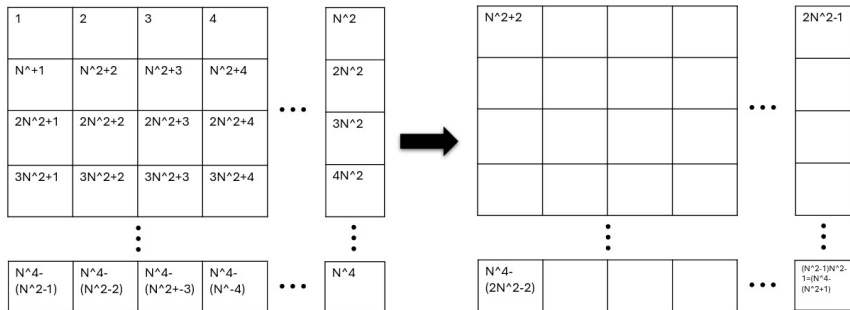
B.C 1

```
Matrix (25 * 25):
Row 00 | 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
```

B.C 2

```
Matrix (25 * 25):
Row 00 | 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
```

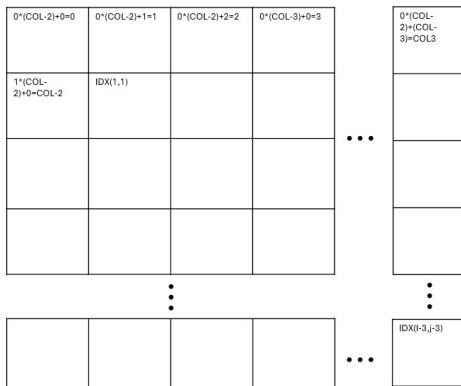

Implementation (CG method)



From $N^2 \times N^2$ to $(N-2) \times (N-2)$

Implementation (CG method)

```
#define IDX(i, j) ((i) * (COL-2) + (j))
```



Implementation (CG method)

B.C 1

```
Matrix (25 * 25):
Row 00 | 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
```

B.C 2

```
Matrix (25 * 25):
Row 00 | 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 -1 0 0 0 0 0 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
```

Implementation (CG method)

B.C 1

```
Matrix (25 * 25):
Row 00 | -4 2 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 2 -5 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 2 -4 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 1 0 0 0 0 -4 2 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 2 -5 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 1 0 0 0 2 -4 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 1 0 0 0 0 -4 2 0 0 0 1 0 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 2 -5 1 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -4 0 0 0 1 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 2 0 0 0 1 0 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -5 1 0 0 0 1 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -4 0 0 0 1 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 2 0 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -5 1 0 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -4 0
```

B.C 2

```
Matrix (25 * 25):
Row 00 | -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 01 | 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 02 | 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 03 | 0 0 1 -4 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 04 | 0 0 0 1 -4 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 05 | 1 0 0 0 0 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 06 | 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
Row 07 | 0 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
Row 08 | 0 0 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
Row 09 | 0 0 0 0 1 0 0 0 2 -4 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
Row 10 | 0 0 0 0 0 1 0 0 0 0 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0
Row 11 | 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0 0 0
Row 12 | 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0 0
Row 13 | 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 0 1 0 0 0 0 0 0
Row 14 | 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0
Row 15 | 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0 0 0 1 0 0 0 0 0
Row 16 | 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0 0
Row 17 | 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0 0
Row 18 | 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 -4 1 0 0 0 1 0 0
Row 19 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -4 0 0 0 1 0 0
Row 20 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0 0 0 0
Row 21 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -5 1 0 0 0 0
Row 22 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0 0
Row 23 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 -4 1 0 0
Row 24 | 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 2 -4 0
```

Implementation (CG method)

Formulate b

```
if (BC==1){
    for (i=0;i<ROW-2;i++){
        for (j=0;j<COL-2;j++){
            b[IDX(i,j)] = dx*dx*func(i+1,j+1,dx,dy);
        }
    }
}
else if(BC==2){
    for (i=0;i<ROW-2;i++){
        for (j=0;j<COL-2;j++){
            b[IDX(i,j)] = dx*dx*func(i+1,j+1,dx,dy);
            if(i==0) {b[IDX(i,j)] += (-1)*-1/(2*pow(pi,2))*func(i,j+1,dx,dy);}
            if(i==ROW-3){b[IDX(i,j)] += (-1)*-1/(2*pow(pi,2))*func(i+2,j+1,dx,dy);}
            if(j==0) {b[IDX(i,j)] += (-1)*-1/(2*pow(pi,2))*func(i+1,j,dx,dy);}
            if(j==COL-3){b[IDX(i,j)] += (-1)*-1/(2*pow(pi,2))*func(i+1,j+2,dx,dy);}
        }
    }
}
```

Implementation (CG method)

Matrix-free A

B.C 1

```
if (BC==1){
    if (size<23){
        for (i=0;i<ROW-2;i++){
            double sac_0 = 1.0+dx*(epi*epi-1)*sin(i*dx*pi)*epi*dx/2;
            double sac_1 = 1.0+dx*(epi*epi-1)*sin(i*dx*pi)*(1-epi)*dx/2;
            for (j=0;j<COL-2;j++){
                double l = -4*x[IDX(i,j)];
                if(i>0) {l+=x[IDX(i-1,j)];}
                if(i<ROW-3){l+=x[IDX(i+1,j)];}
                if(j>0) {l+=x[IDX(i,j-1)];}
                if(j<COL-3){l+=x[IDX(i,j+1)];}
                if (j==1) {l-=sac_0*x[IDX(i,j)];l+=x[IDX(i,j-1)];}
                if (j==COL-2){l-=sac_1*x[IDX(i,j)];l+=x[IDX(i,j+1)];}
                b[IDX(i,j)]=l;
            }
        }
    }
    else{
        for (i=0;i<ROW-2;i++){
            for (j=0;j<COL-2;j++){
                double l = -4*x[IDX(i,j)];
                if(i>0) {l+=x[IDX(i-1,j)];}
                if(i<ROW-3){l+=x[IDX(i+1,j)];}
                if(j>0) {l+=x[IDX(i,j-1)];}
                if(j<COL-3){l+=x[IDX(i,j+1)];}
                b[IDX(i,j)]=l;
            }
        }
    }
}
```

B.C 2

```
else if(BC==2){
    for (i=0;i<ROW-2;i++){
        for (j=0;j<COL-2;j++){
            double l = -4*x[IDX(i,j)];
            if(i>0) {l+=x[IDX(i-1,j)];}
            if(i<ROW-3){l+=x[IDX(i+1,j)];}
            if(j>0) {l+=x[IDX(i,j-1)];}
            if(j<COL-3){l+=x[IDX(i,j+1)];}
            b[IDX(i,j)]=l;
        }
    }
}
```

Implementation (CG method)

Zeroth iteration

```
if (use_matrix_A){
    make_Abx(A,b,x,p,dx,dy,BC,ROW,use_matrix_A);
    vmdot(A,x,tmp);
}
else {
    make_Abx(NULL,b,x,p,dx,dy,BC,ROW,use_matrix_A);
    Adot(x,tmp,BC,ROW);
}

for (i=0;i<ROW-2;i++){
    for (j=0;j<COL-2;j++){
        r[IDX(i,j)] = b[IDX(i,j)] - tmp[IDX(i,j)];
        z[IDX(i,j)] = r[IDX(i,j)];
    }
}
```

Implementation (CG method)

Main loop

```
for (it=0; it<itmax; it++){
{
    if(use_matrix_A){vmdot(A,z,tmp);}
    else{Adot(z,tmp,BC,ROW);}
    alpha = vvdot(r,r)/vvdot(z,tmp);

    for (i=0; i<ROW-2; i++){
        for (j=0; j<COL-2; j++){
            x[IDX(i,j)] = x[IDX(i,j)] + alpha * z[IDX(i,j)];
            r_new[IDX(i,j)] = r[IDX(i,j)] - alpha*tmp[IDX(i,j)];
        }
    }
}
```

```
if (norm_L2(r_new) < tol){
//-----
// Redistribute x vector to array
//-----
for (i=0; i<ROW-2; i++){
    for (j=0; j<COL-2; j++){
        p[i+1][j+1] = x[IDX(i,j)];
    }
}
if (BC==1){
    for(int i=0; i<COL; i++){
        p[i][0] = p[i][1];
        p[i][COL-1] = p[i][COL-2];
    }
}
else if (BC==2){
    for(int i=0; i<ROW; i++){
        p[i][0] = -1/(2*pow(pi,2))*func(i,0,dx,dy);
        p[i][COL-1] = -1/(2*pow(pi,2))*func(i,COL-1,dx,dy);
    }
    for(int j=0; j<COL; j++){
        p[0][j] = -1/(2*pow(pi,2))*func(0,j,dx,dy);
        p[ROW-1][j] = -1/(2*pow(pi,2))*func(ROW-1,j,dx,dy);
    }
}

*iter = it;
if (use_matrix_A){
    for (int i = 0; i < (ROW-2)*(COL-2); i++){
        free(A[i]);
    }
    free(A);
}
free(tmp);
```


Implementation (CG method)

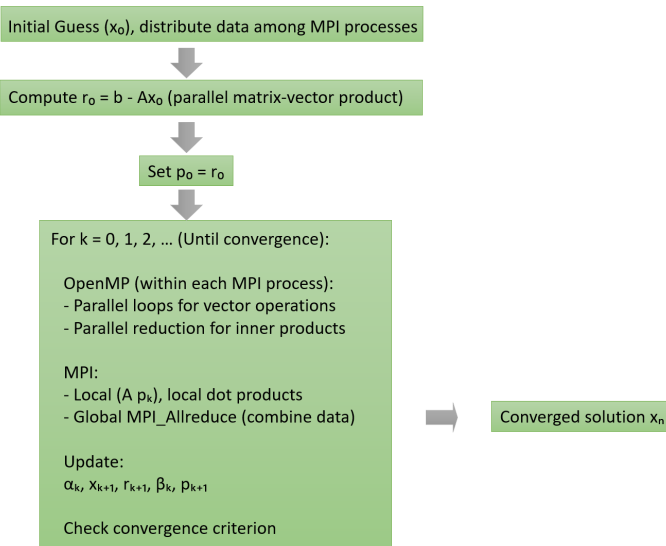
```
beta = vvdot(r_new,r_new)/vvdot(r,r);  
for (i=0;i<ROW-2;i++){  
    for (j=0;j<COL-2;j++){  
        z[IDX(i,j)] = r_new[IDX(i,j)] + beta*z[IDX(i,j)];  
        r[IDX(i,j)] = r_new[IDX(i,j)];  
    }  
}
```

- Parallelizable Operations
 - Shared Memory Environment: All threads access shared data structures (matrix and vectors).
 - Parallel Matrix-Vector Multiplication: Distribute loop iterations evenly across threads.
 - Parallel Reduction for Inner Products: Sum computed in parallel using OpenMP reduction clauses.
 - Synchronization: OpenMP implicitly synchronizes threads at the end of parallel regions.

CG Algorithm Parallelization with MPI

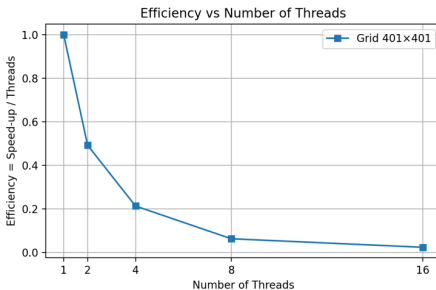
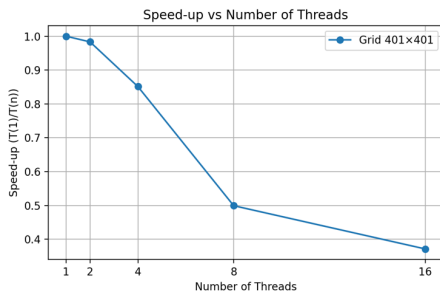
- MPI Algorithm Steps
 - Distribute Matrix and Vectors among MPI processes.
 - Local Computation: Each MPI process computes local parts of matrix-vector products and inner products.
 - Global Reduction: MPI_Allreduce synchronizes and sums local computations for global values (e.g., residual norm, dot products).
 - Iteration Update: Update vectors x, r, p_x, r, p independently after global synchronization.

Schematics of Parallelization



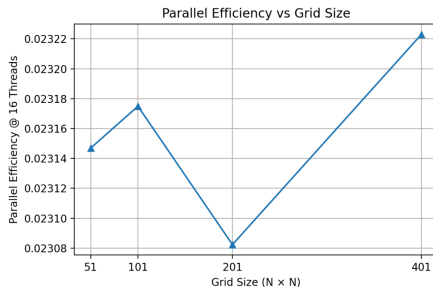
Result of OpenMP

- False sharing / cache contention
- Insufficient computational workload per thread
- Overhead of thread management
- Critical sections or reduction barriers.
- No data locality or cache blocking
- Too much parallel overhead for this problem size



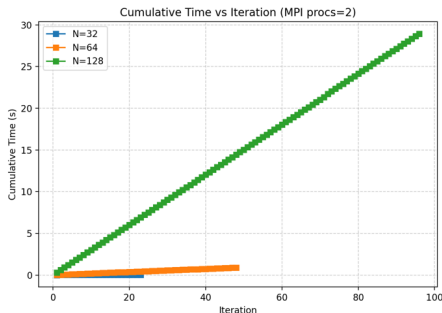
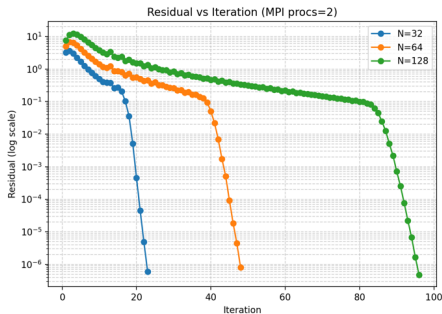
Result of OpenMP

- Low Efficiency at Grid Size 201×201
 - Load Imbalance
 - Cache and Memory Issues
 - Synchronization Overhead



Result of MPI

- A Larger grid means more unknowns and a more complex linear system.
- CG convergence is affected by the condition number of matrix A, which increases with grid size, slowing convergence.
- More data to distribute and synchronize across MPI processes.
- Larger N implies matrix-vector products scale as $O(N^2)$ to $O(N^3)$ depending on implementation.



Performance with Difference BC

- When the grid size N is **small**, using **boundary condition 1** results in a larger number of CG iterations. This is due to oscillations caused by the **asymmetry** of the matrix associated with BC1.

```
caiyilun@caiyilunde-MacBook-Pro code % ./final_project 1 false 10 10
設定後 Nx : 10, Ny : 10
mkdir: RESULT: File exists
```

```
-----
Nx : 10, Ny : 10
Tolerance : 1.000000e-08, Omega : 1.800000
```

```
[CG Method]
CG_1 method - Error : 6.433132e-04, Iteration : 2563, Time : 0.001827 s
```

```
[SOR Method]
SOR_1 Method - Error : 3.469912e-04, Iteration : 61, Time : 0.000132 s
caiyilun@caiyilunde-MacBook-Pro code %
```

```
if (argc >= 2) BC = atoi(argv[1]);
if (argc >= 3) use_matrix_A = (strcmp(argv[2], "true") == 0);
if (argc >= 4) ROW = atoi(argv[3]);
if (argc >= 5) COL = atoi(argv[4]);
```

Difference BC

```
caiyilun@caiyilunde-MacBook-Pro code % ./final_project 2 false 10 10
設定後 Nx : 10, Ny : 10
mkdir: RESULT: File exists
```

```
-----
Nx : 10, Ny : 10
Tolerance : 1.000000e-08, Omega : 1.800000
```

```
[CG Method]
CG_2 method - Error : 7.908507e-06, Iteration : 7, Time : 0.000040 s
```

```
[SOR Method]
SOR_2 Method - Error : 7.908508e-06, Iteration : 85, Time : 0.000333 s
caiyilun@caiyilunde-MacBook-Pro code %
```


Performance with Difference Grid Size N

- As the number of grid points **increases**, the number of iterations rises significantly. At the same time, the influence of boundary conditions on the Conjugate Gradient (CG) method gradually diminishes, the **accuracy of the numerical solution improves**, but this also comes with a **longer computation time**.

```
-----
Nx : 50, Ny : 50
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 2.257441e-05, Iteration : 170, Time : 0.004157 s

[SOR Method]
SOR_1 Method - Error : 1.095029e-05, Iteration : 674, Time : 0.042238 s
caiyilun@caiyilunde-MacBook-Pro code % █

-----
Nx : 100, Ny : 100
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 0.023904 s

[SOR Method]
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.596733 s
caiyilun@caiyilunde-MacBook-Pro code % █

-----
Nx : 200, Ny : 200
Tolerance : 1.000000e-08, Omega : 1.800000
-----

[CG Method]
CG_1 method - Error : 1.325926e-06, Iteration : 441, Time : 0.175544 s

[SOR Method]
SOR_1 Method - Error : 6.571365e-07, Iteration : 6800, Time : 7.152766 s
caiyilun@caiyilunde-MacBook-Pro code % █
```

Performance with and w/o Matrix-Free

- Enabling the matrix-free mode eliminates the cost of explicitly constructing the sparse matrix, which **significantly improves computational performance and reduces memory usage**, especially for large-scale grids. Compared to the CG method with an explicit matrix, the matrix-free version achieves noticeably **faster runtime**.
- In contrast, the SOR method is unaffected by this setting since it does not rely on a global matrix structure. Overall, the matrix-free approach is particularly well-suited for solving large-scale problems using the CG method.

```
-----  
Nx : 100, Ny : 100  
Tolerance : 1.000000e-08, Omega : 1.800000  
-----
```

With Matrix-Free

```
[CG Method]  
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 0.020315 s  
  
[SOR Method]  
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.554946 s  
caiyilun@caiyilunde-MacBook-Pro code % █
```

```
-----  
Nx : 100, Ny : 100  
Tolerance : 1.000000e-08, Omega : 1.800000  
-----
```

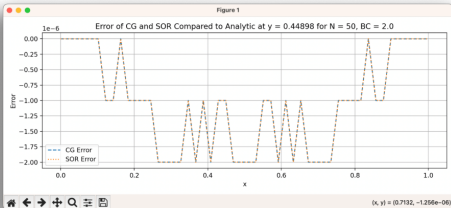
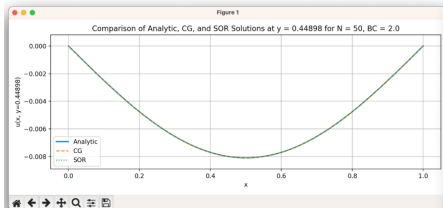
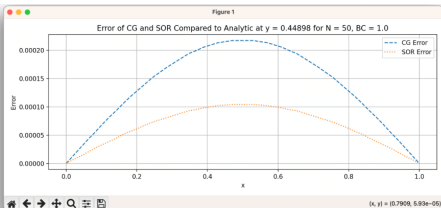
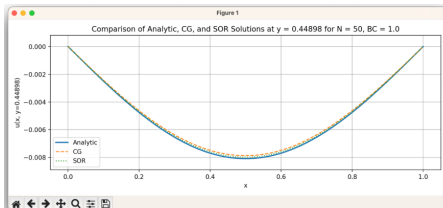
Without Matrix-Free

```
[CG Method]  
CG_1 method - Error : 5.418124e-06, Iteration : 222, Time : 26.813761 s  
  
[SOR Method]  
SOR_1 Method - Error : 2.664048e-06, Iteration : 2193, Time : 0.531892 s  
caiyilun@caiyilunde-MacBook-Pro code % █
```

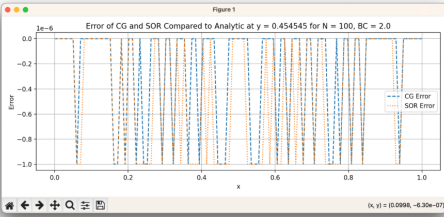
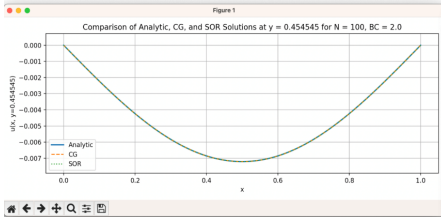
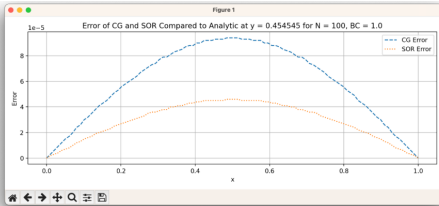
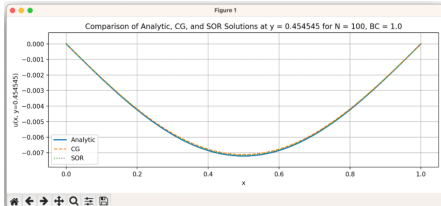
Wall-Clock Time vs. Number of cells

	SOR		CG (with matrix-free)		CG (without matrix-free)	
	Error	Time	Error	Time	Error	Time
N = 50	1.0950e-05	0.041171 s	2.2574e-05	0.004034 s	2.2574e-05	1.162133 s
N = 100	2.6641e-06	0.554324 s	5.4181e-06	0.023376 s	5.4181e-06	27.224622 s
N = 200	6.5714e-07	6.953978 s	1.3259e-06	0.177531 s	none	none

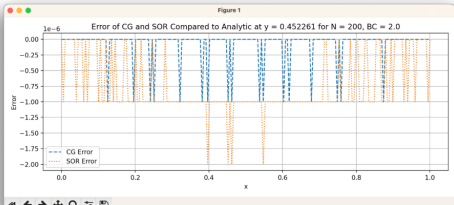
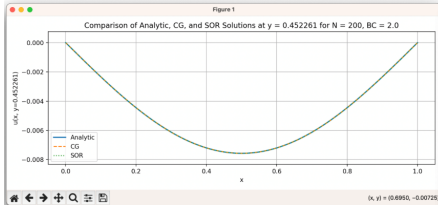
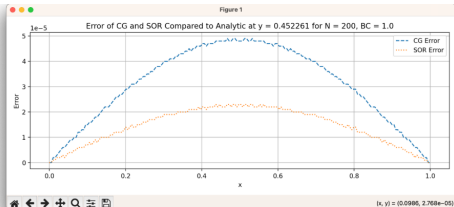
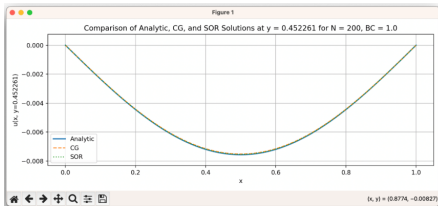
Performance (other plots)



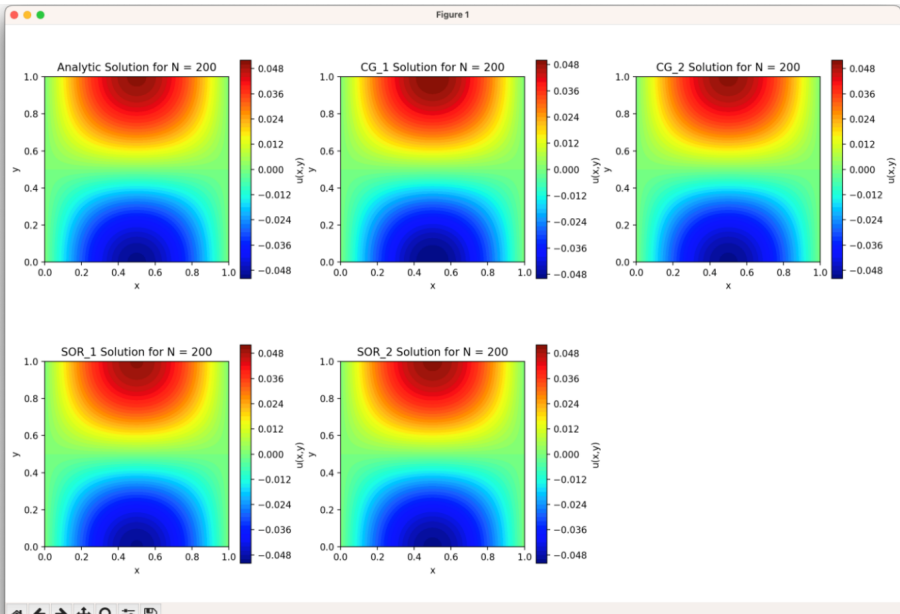
Performance (other plots)



Performance (other plots)



Performance (other plots)



- Q&A